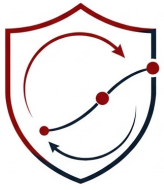
Principles of Robot Autonomy I

Planning and Control: Trajectory following, feedback control

Stanford University



SIA Lab
Safe and Intelligent Autonomy Lab

IPRL

Principles of Robot Autonomy I

Announcements:

- Lab section 2 this week, in person
- HW 1 due today, Thurs Oct 8 at 11:59pm
- HW 2 out today, due Thurs, Oct 15 at 11:59pm
- Some changes in the section schedule. Staff email list: cs237a-aut2627-staff@lists.stanford.edu
- If your local ROS install not working: Come to **any OHs or any lab section.** We'll have extra laptops with ROS and CAs to help.
- Enrolled: 170, Waitlist: 11

Lecture 6:

- Spline trajectory optimization
- Differential flatness planning
- Trajectory following, feedback control
- Diff drive robot trajectory following
- Differential flatness
- LQR, gain scheduled LQR

Stanford University

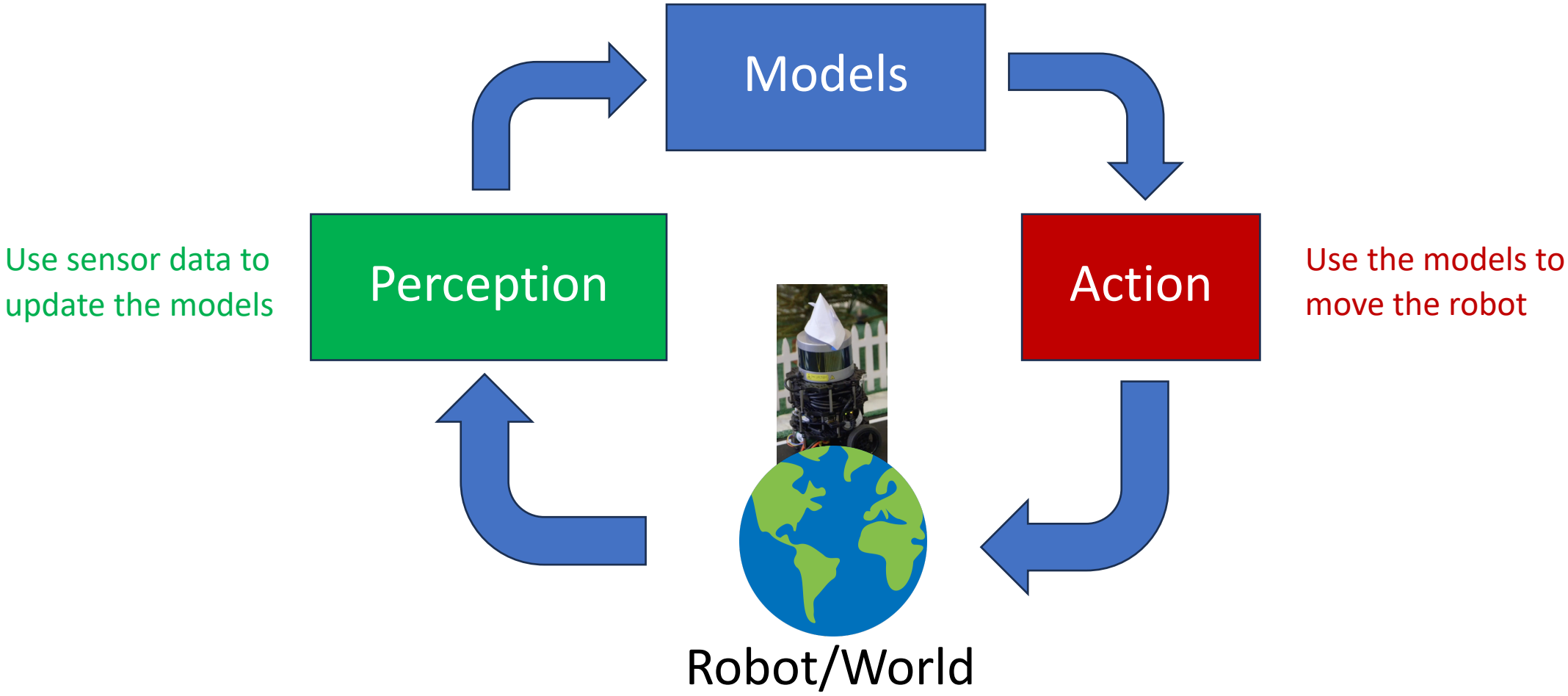
SIA Lab
Safe and Intelligent Autonomy Lab

MSL

IPRL

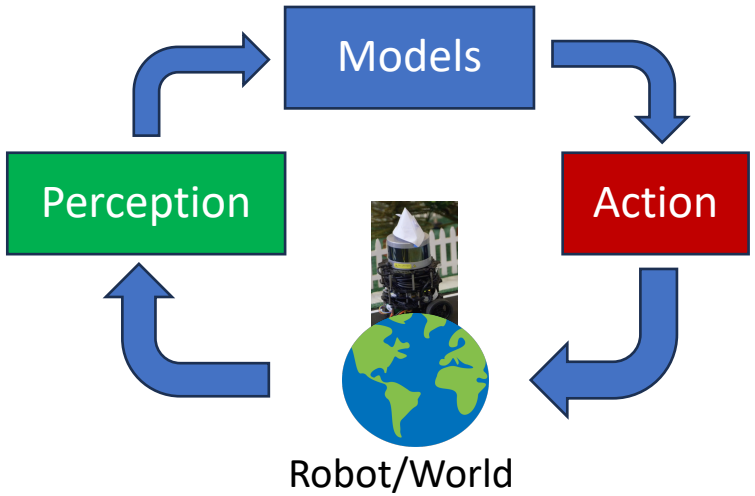
ASL
Autonomous Systems Lab

Full Stack Autonomy: the Perception-Action Loop



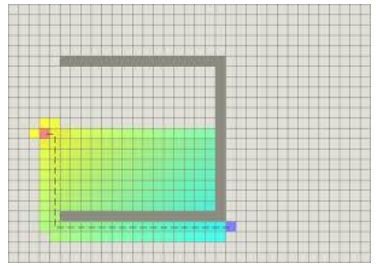
Action: Planning and Control Stack

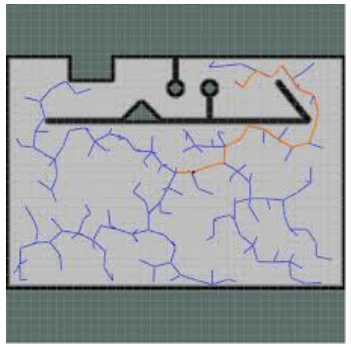
Use the models to move the robot



Path planning:

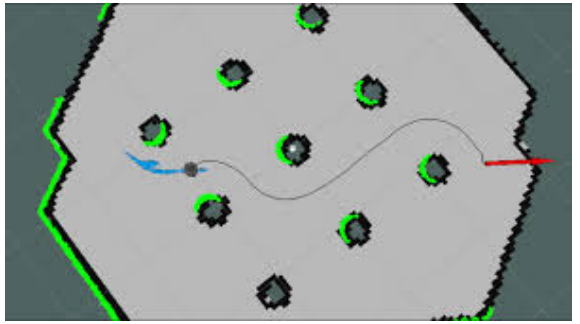
- A* planning on a graph
- RRT, RRT*





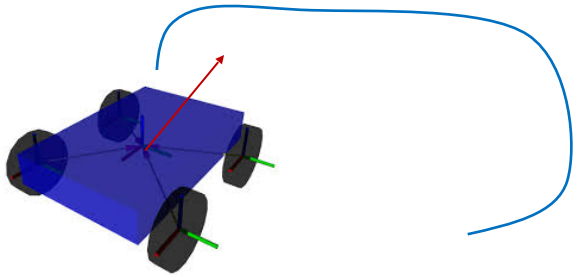
Trajectory optimization:

- Optimal control
- Numerical solvers



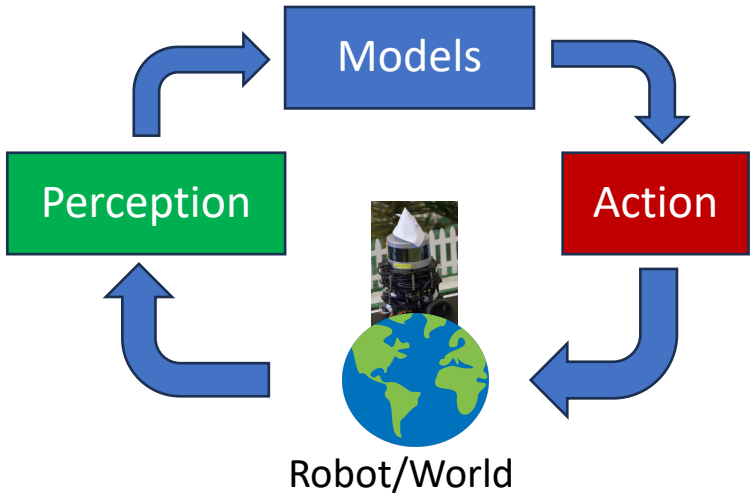
Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



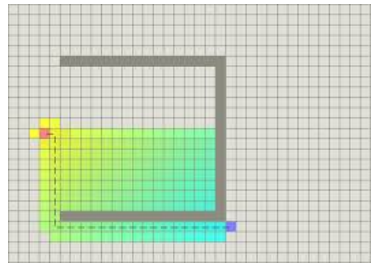
Action: Planning and Control Stack

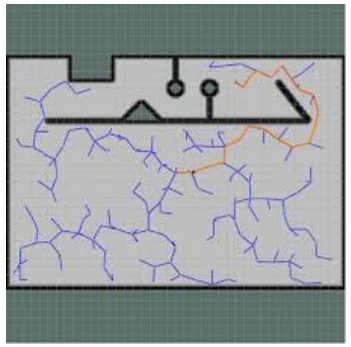
Use the models to move the robot



Path planning:

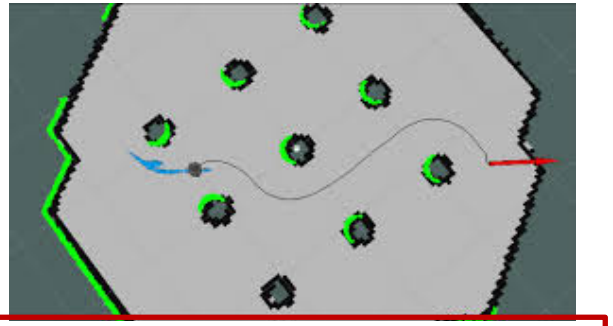
- A* planning on a graph
- RRT, RRT*





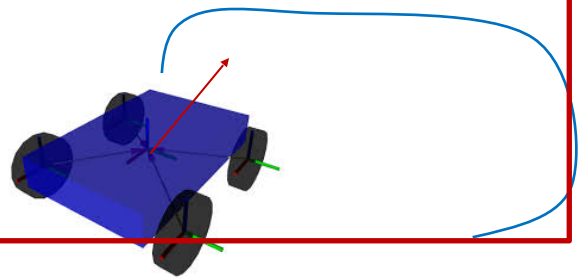
Trajectory optimization:

- Optimal control
- Numerical solvers

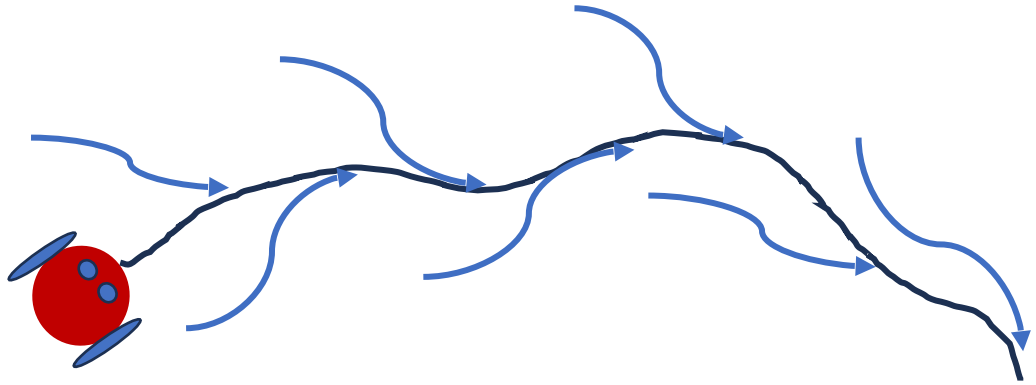


Trajectory following:

- Kinematic/dynamic models
- PID control
- LQR, gain scheduled LQR



Trajectory Following



Motion model:

$$x_{t+1} = f(x_t, u_t)$$

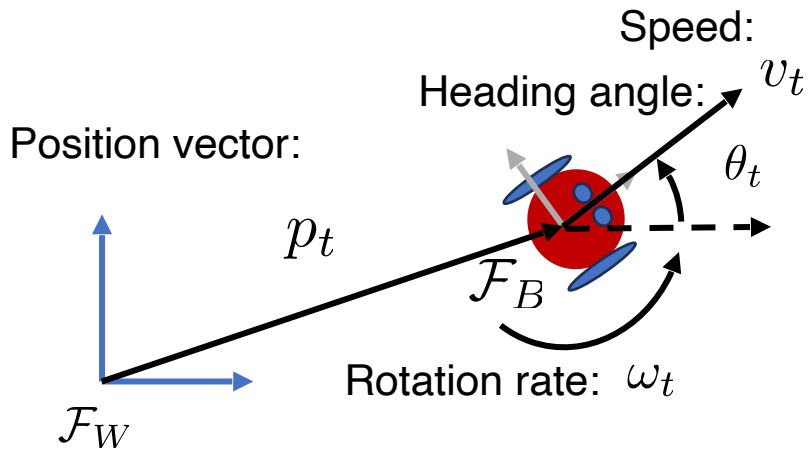
Feedback control law:

$$u_t = h(x_t, x_t^*)$$

planned traj waypoints

Goal: Use closed loop feedback control to track desired trajectory, rejecting disturbances and being robust to model errors.

Robot Kinematics and Dynamics



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

Differential Drive Robot (kinematic):

$$
\begin{aligned}
x^p_{t+1} &= x^p_t + v_t \cos(\theta_t)\delta t \\
y^p_{t+1} &= y^p_t + v_t \sin(\theta_t)\delta t \\
\theta_{t+1} &= \theta_t + \omega_t \delta t
\end{aligned}
$$

Nonholonomic: constraint on velocity vector
Nonlinear

Robot dynamic state: $x_t = (p_t, \theta_t, v_t, \omega_t)$

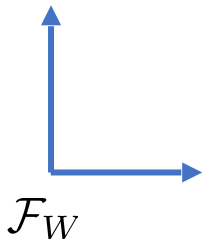
Control inputs: $u_t = (a_t, \alpha_t)$

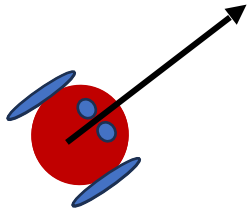
Differential Drive Robot (dynamic):

$$
\begin{aligned}
x^p_{t+1} &= x^p_t + v_t \cos(\theta_t)\delta t \\
y^p_{t+1} &= y^p_t + v_t \sin(\theta_t)\delta t \\
\theta_{t+1} &= \theta_t + \omega_t \delta t \\
v_{t+1} &= v_t + a_t \delta t \\
\omega_{t+1} &= \omega_t + \alpha_t \delta t
\end{aligned}
$$

Heuristic Proportional Controller

Position vector:





Robot pose: (p_t, θ_t)

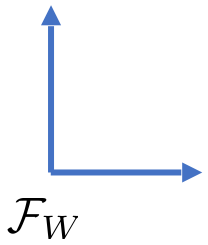
Control inputs: (v_t, ω_t)

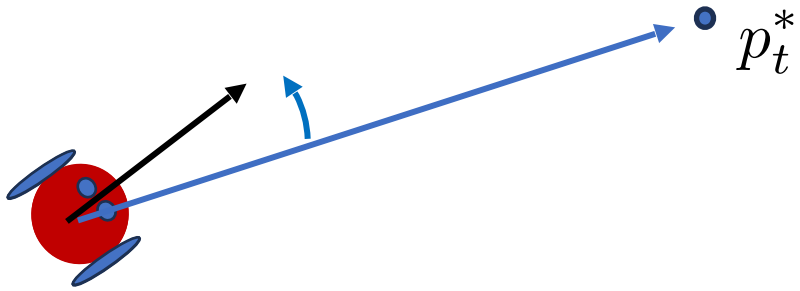
Differential Drive Robot (kinematic):

$$
\begin{aligned}
x^p_{t+1} &= x^p_t + v_t \cos(\theta_t)\delta t \\
y^p_{t+1} &= y^p_t + v_t \sin(\theta_t)\delta t \\
\theta_{t+1} &= \theta_t + \omega_t \delta t
\end{aligned}
$$

Heuristic Proportional Controller

Position vector:





Robot pose: (p_t, θ_t)

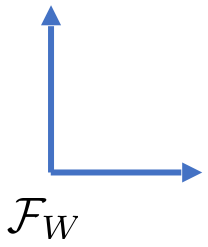
Control inputs: (v_t, ω_t)

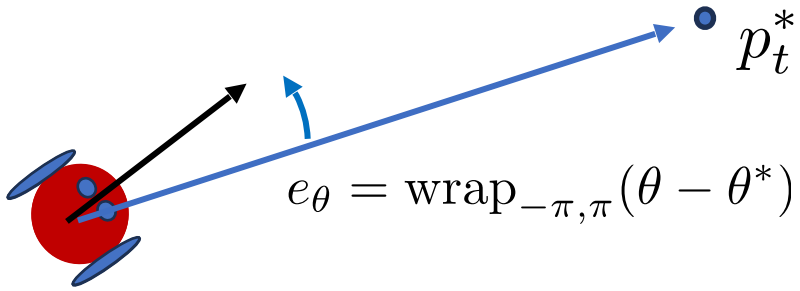
Differential Drive Robot (kinematic):

$$
\begin{aligned}
x_{t+1}^p &= x_t^p + v_t \cos(\theta_t)\delta t \\
y_{t+1}^p &= y_t^p + v_t \sin(\theta_t)\delta t \\
\theta_{t+1} &= \theta_t + \omega_t \delta t
\end{aligned}
$$

Heuristic Proportional Controller

Position vector:





Robot pose: (p_t, θ_t)

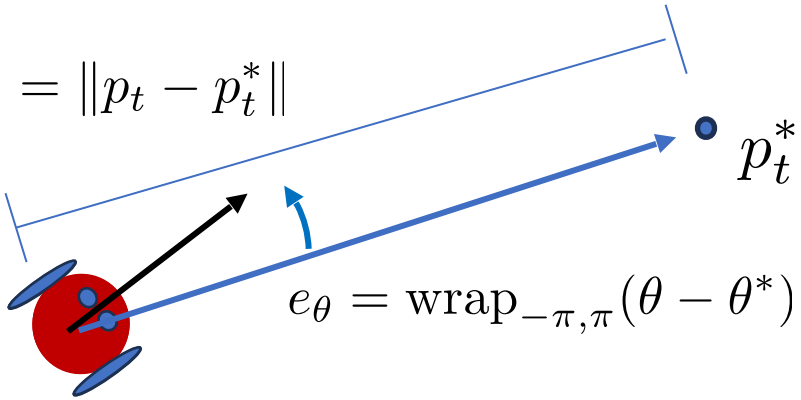
Control inputs: (v_t, ω_t)

$$\omega = -k_\theta e_\theta$$

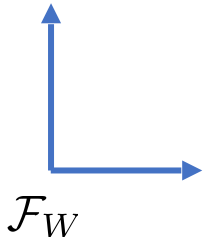
Differential Drive Robot (kinematic):

$$\begin{aligned} x^p_{t+1} &= x^p_t + v_t \cos(\theta_t)\delta t \\ y^p_{t+1} &= y^p_t + v_t \sin(\theta_t)\delta t \\ \theta_{t+1} &= \theta_t + \omega_t \delta t \end{aligned}$$

Heuristic Proportional Controller



Position vector:



Robot pose: (p_t, θ_t)

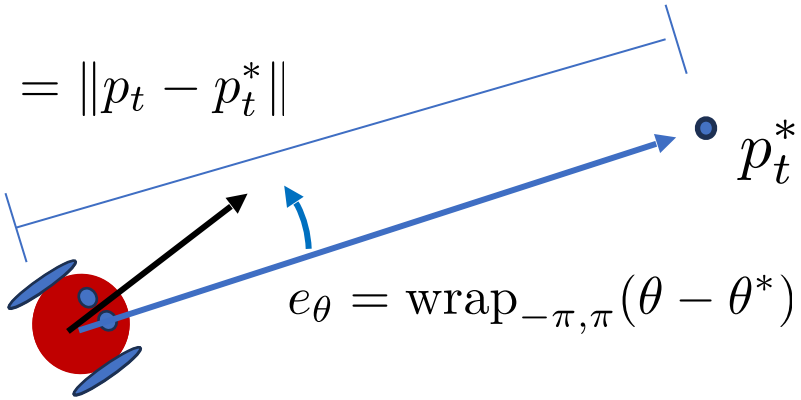
Control inputs: (v_t, ω_t)

$$\omega = -k_\theta e_\theta$$

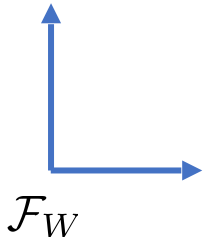
Differential Drive Robot (kinematic):

$$x_{t+1}^p = x_t^p + v_t \cos(\theta_t)\delta t$$
$$y_{t+1}^p = y_t^p + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

Heuristic Proportional Controller



Position vector:



Robot pose: (p_t, θ_t)

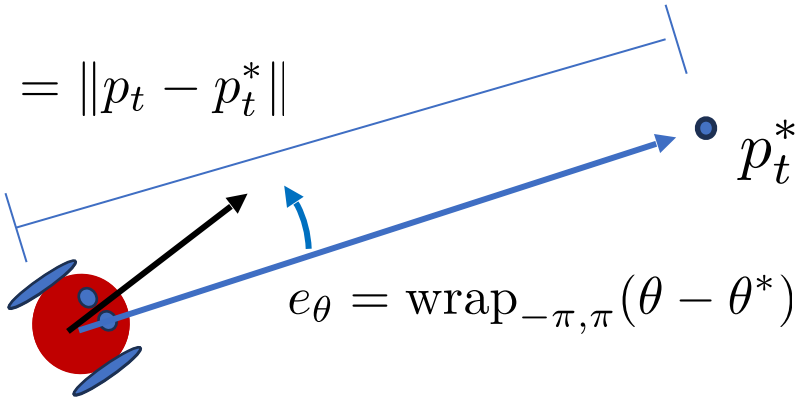
Control inputs: (v_t, ω_t)

$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

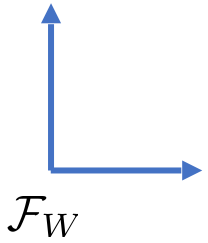
Differential Drive Robot (kinematic):

$$x_{t+1}^p = x_t^p + v_t \cos(\theta_t)\delta t$$
$$y_{t+1}^p = y_t^p + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

Heuristic Proportional Controller



Position vector:



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

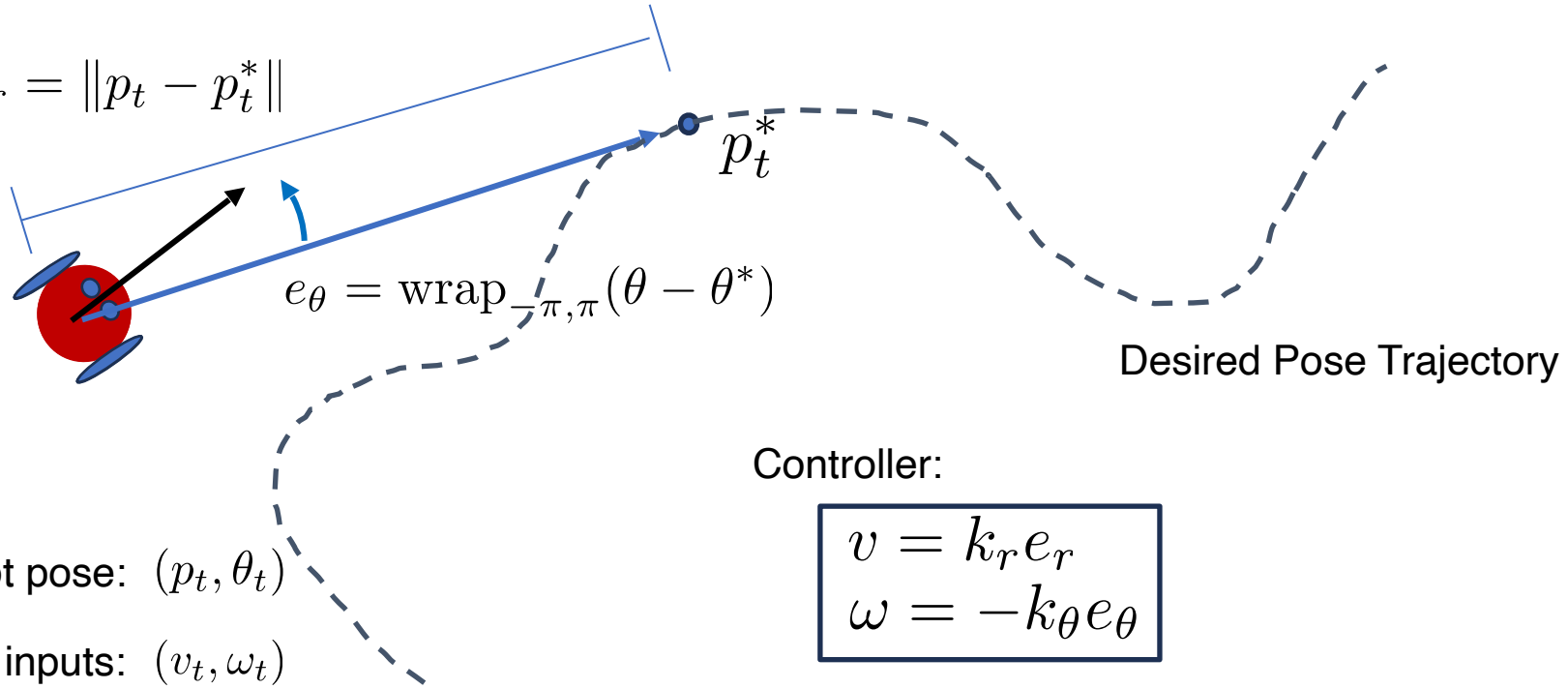
Controller:

$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

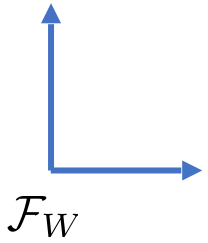
Differential Drive Robot (kinematic):

$$x^p_{t+1} = x^p_t + v_t \cos(\theta_t)\delta t$$
$$y^p_{t+1} = y^p_t + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

Heuristic Proportional Controller



Position vector:



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

Controller:

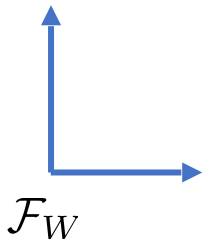
$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

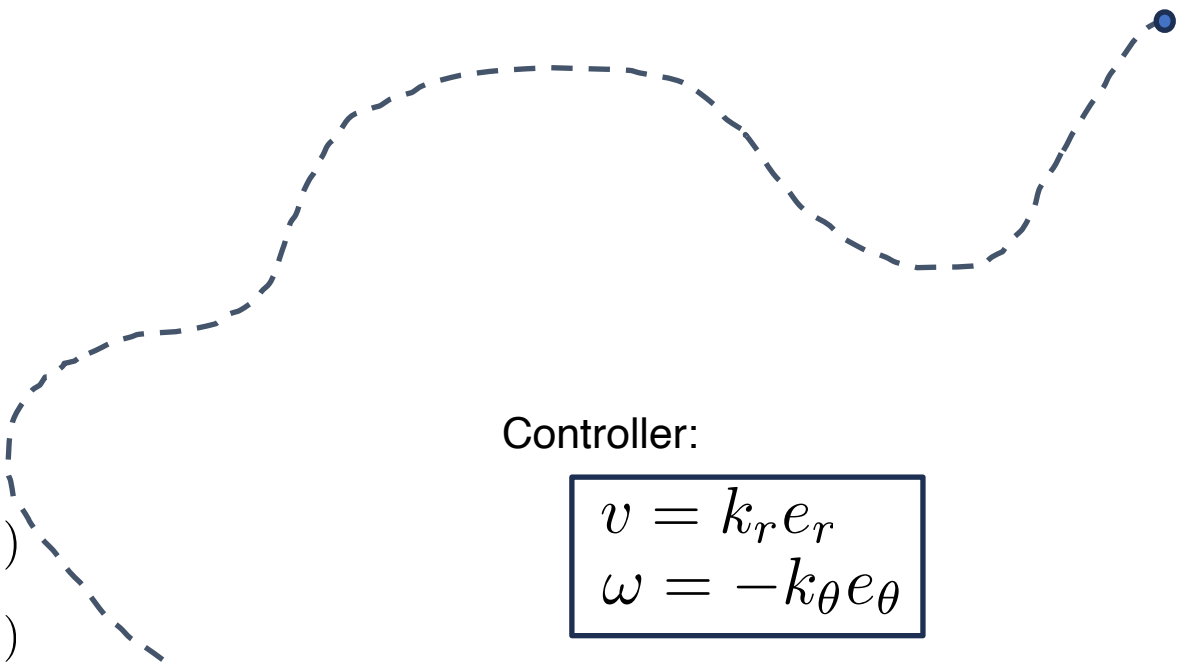
Differential Drive Robot (kinematic):

$$x_{t+1}^p = x_t^p + v_t \cos(\theta_t)\delta t$$
$$y_{t+1}^p = y_t^p + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

What about heading?

Position vector:





Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

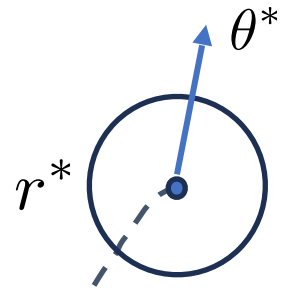
Controller:

$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

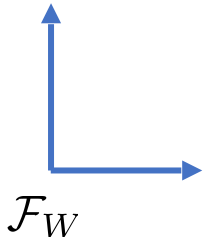
Differential Drive Robot (kinematic):

$$x^p_{t+1} = x^p_t + v_t \cos(\theta_t)\delta t$$
$$y^p_{t+1} = y^p_t + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

What about heading?



Position vector:



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

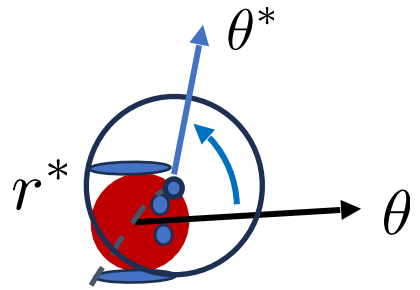
Controller:

$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

Differential Drive Robot (kinematic):

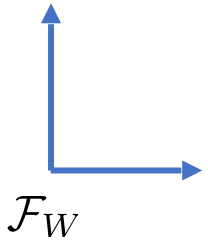
$$x^p_{t+1} = x^p_t + v_t \cos(\theta_t)\delta t$$
$$y^p_{t+1} = y^p_t + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

What about heading?



$$e_\theta = \text{wrap}_{[-\pi,\pi]}(|\theta - \theta^*|)$$

Position vector:



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

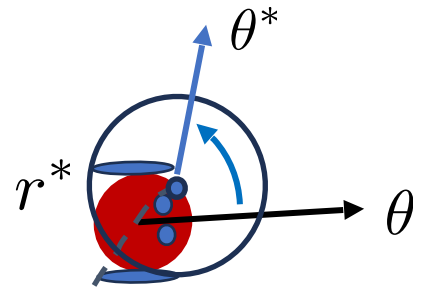
Controller:

$$v = k_r e_r$$
$$\omega = -k_\theta e_\theta$$

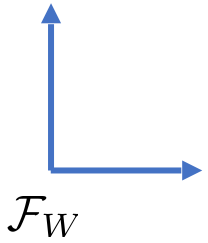
Differential Drive Robot (kinematic):

$$x^p_{t+1} = x^p_t + v_t \cos(\theta_t)\delta t$$
$$y^p_{t+1} = y^p_t + v_t \sin(\theta_t)\delta t$$
$$\theta_{t+1} = \theta_t + \omega_t \delta t$$

What about heading?



Position vector:



Robot pose: (p_t, θ_t)

Control inputs: (v_t, ω_t)

$$e_\theta = \text{wrap}_{[-\pi,\pi]}(|\theta - \theta^*|)$$

Controller:

$$\begin{aligned} v &= k_r e_r \\ \omega &= -k_\theta e_\theta \end{aligned}$$

Switching rule:

$$\begin{aligned} \text{if} \quad & \|p_t - p_t^*\| \leq r^* \\ & \theta^* = \text{desired heading} \\ & k_r = 0 \end{aligned}$$

Differential Drive Robot (kinematic):

$$\begin{aligned} x_{t+1}^p &= x_t^p + v_t \cos(\theta_t)\delta t \\ y_{t+1}^p &= y_t^p + v_t \sin(\theta_t)\delta t \\ \theta_{t+1} &= \theta_t + \omega_t \delta t \end{aligned}$$

Isn’t there an easier way?

Unfortunately, no.

Brockett’s Theorem (1983): There exists no **continuously differentiable** feedback control law to stabilize a nonholonomic system to a desired pose.

We can only achieve desired position and heading with a switching law.

Many other switching controllers for diff drive robot exist in literature.

Feedback Linearizing Controller

Differential Drive Robot (continuous time):

$$\begin{aligned} \dot{x}^p &= v_t \cos\theta \\ \dot{y}^p &= v_t \sin\theta \\ \dot{\theta} &= \omega \end{aligned}$$

Differentiate velocity wrt time:

$$\begin{aligned} \ddot{x}^p &= \dot{v}\cos(\theta) - v\sin(\theta)\dot{\theta} \\ \ddot{y}^p &= \dot{v}\sin(\theta) + v\cos(\theta)\dot{\theta} \\ \dot{v} &= a \\ \dot{\theta} &= \omega \end{aligned}$$

$$\begin{bmatrix} \ddot{x}^p \\ \ddot{y}^p \end{bmatrix} = \underbrace{\begin{bmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{bmatrix}}_{J(v,\theta)} \begin{bmatrix} a \\ \omega \end{bmatrix}$$

We design controller as:

$$\begin{bmatrix} a \\ \omega \end{bmatrix} = J^{-1} f_{des}(x^p, y^p, \dot{x}^p, \dot{y}^p)$$

Closed loop dynamics:

$$\begin{bmatrix} \ddot{x}^p \\ \ddot{y}^p \end{bmatrix} = f_{des}(x^p, y^p, \dot{x}^p, \dot{y}^p)$$

We choose f_{des} to give dynamics we like, e.g., linear.

Feedback Linearizing Controller

Dynamics:

$$\ddot{p} = Ju$$

Controller:

$$u = J^{-1}\underbrace{(\ddot{p}^* + k_d(\dot{p}^* - \dot{p}) + k_p(p^* - p))}_{f_{des}}$$

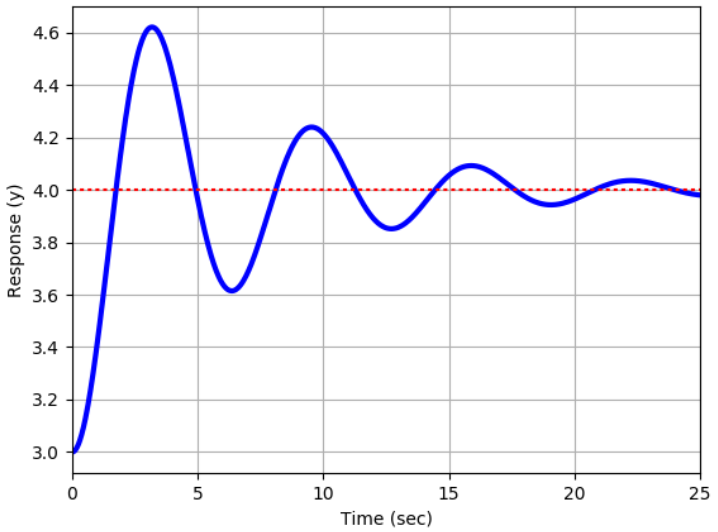
Closed loop dynamics:

$$\ddot{p} = \ddot{p}^* + k_d\underbrace{(\dot{p}^* - \dot{p})}_{-\delta\dot{p}} + k_p\underbrace{(p^* - p)}_{-\delta p}$$

Rewrite as:

$$\delta\ddot{p} + k_d(\delta\dot{p}) + k_p(\delta p) = 0$$

Exponential convergence: $p(t) \to p^*(t)$



Practical Details

Controller:

$$u = J^{-1}(\ddot{p}^* + k_d(\dot{p}^* - \dot{p}) + k_p(p^* - p))$$

where: $J = \begin{bmatrix} \cos\theta & -v\sin\theta \\ \sin\theta & v\cos\theta \end{bmatrix}$

Singularity at $v = 0$

- We couple this with the switching rule we saw before (Brockett's theorem is unavoidable)
- We building in safety checks to avoid $v \leq v_{\text{lower bound}}$
- Need to discretize in time to implement

LQR Based Controllers

The LQR problem:

Quadratic state cost

Quadratic control cost

$$\min_u \int_0^\infty x^T Q x + u^T R u \, dt$$

$$\text{subject to} \quad \dot{x} = Ax + Bu$$

Linear dynamics

$$x(0) = x_0$$

Initial conditions

LQR Solution

Open-loop system dynamics:

$$\dot{x} = Ax + Bu$$

Linear feedback controller:

$$u = -Kx$$

LQR Solution

Open-loop system dynamics:

$$\dot{x} = Ax + Bu$$

Linear feedback controller:

$$u = -Kx$$

Solve Riccati equation for P:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

Obtain K from:

$$K = R^{-1}B^T P$$

LQR Solution

Open-loop system dynamics:

$$\dot{x} = Ax + Bu$$

Closed loop system:

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

Linear feedback controller:

$$u = -Kx$$

Solve Riccati equation for P:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

Obtain K from:

$$K = R^{-1}B^T P$$

LQR Solution

Open-loop system dynamics:

$$\dot{x} = Ax + Bu$$

Linear feedback controller:

$$u = -Kx$$

Solve Riccati equation for P:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

Obtain K from:

$$K = R^{-1}B^T P$$

Closed loop system:

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

Closed loop dynamics matrix determines convergence rate, response modes:

$$A_{cl} = (A - BK)$$

LQR Solution

Open-loop system dynamics:

$$\dot{x} = Ax + Bu$$

Linear feedback controller:

$$u = -Kx$$

Solve Riccati equation for P:

$$A^T P + PA - PBR^{-1}B^T P + Q = 0$$

Obtain K from:

$$K = R^{-1}B^T P$$

Closed loop system:

$$\dot{x} = Ax + B(-Kx) = (A - BK)x$$

Closed loop dynamics matrix determines convergence rate, response modes:

$$A_{cl} = (A - BK)$$

Advantages:

- Automatic control design
- Performs very well in practice
- Computationally very light
- Work for arbitrary dimensional systems

Challenges:

- Requires linear dynamics!
- Only regulates state to 0!
- How to adapt to nonlinear dynamics and tracking objectives?

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \nabla_x f(x^*, u^*)(x - x^*) + \nabla_u f(x^*, u^*)(u - u^*)$$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))}(x - x^*) + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))}(u - u^*)$$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))}\underbrace{(x - x^*)}_{\delta x} + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))}\underbrace{(u - u^*)}_{\delta u}$$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))} \underbrace{(x - x^*)}_{\delta x} + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))} \underbrace{(u - u^*)}_{\delta u}$$

We get: $\boxed{\delta\dot{x} = A(t)\delta x + B(t)\delta u}$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))}\underbrace{(x - x^*)}_{\delta x} + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))}\underbrace{(u - u^*)}_{\delta u}$$

$$\underbrace{(\dot{x} - \dot{x}^*)}_{\delta \dot{x}} = A(t)\delta x + B(t)\delta u$$

We get: $\boxed{\delta \dot{x} = A(t)\delta x + B(t)\delta u}$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

Recall Jacobians:

$$\nabla_x f(x^*, u^*) = \begin{bmatrix} \frac{\partial f_1(x,u)}{\partial x_1}\Big|_{x^*,u^*} & \cdots & \frac{\partial f_1(x,u)}{\partial x_n}\Big|_{x^*,u^*} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n(x,u)}{\partial x_1}\Big|_{x^*,u^*} & \cdots & \frac{\partial f_n(x,u)}{\partial x_n}\Big|_{x^*,u^*} \end{bmatrix}$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))}\underbrace{(x - x^*)}_{\delta x} + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))}\underbrace{(u - u^*)}_{\delta u}$$

$$\underbrace{(\dot{x} - \dot{x}^*)}_{\delta \dot{x}} = A(t)\delta x + B(t)\delta u$$

We get: $\boxed{\delta \dot{x} = A(t)\delta x + B(t)\delta u}$

Taylor Series Linearization

Open-loop nonlinear system dynamics:

$$\dot{x} = f(x, u)$$

Linearize about desired **feasible trajectory** from traj opt**:**

$$(x^*(t), u^*(t)) \quad \text{such that} \quad \dot{x}^*(t) = f(x^*(t), u^*(t))$$

1st order Taylor series expantion:

$$\dot{x} \approx f(x^*, u^*) + \underbrace{\nabla_x f(x^*, u^*)}_{A(x^*(t), u^*(t))}\underbrace{(x - x^*)}_{\delta x} + \underbrace{\nabla_u f(x^*, u^*)}_{B(x^*(t), u^*(t))}\underbrace{(u - u^*)}_{\delta u}$$

$$\underbrace{(\dot{x} - \dot{x}^*)}_{\delta \dot{x}}$$

We get:

$$\boxed{\delta \dot{x} = A(t)\delta x + B(t)\delta u}$$

Recall Jacobians:

$$\nabla_x f(x^*, u^*) = \begin{bmatrix} \frac{\partial f_1(x,u)}{\partial x_1}|_{x^*,u^*} & \cdots & \frac{\partial f_1(x,u)}{\partial x_n}|_{x^*,u^*} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n(x,u)}{\partial x_1}|_{x^*,u^*} & \cdots & \frac{\partial f_n(x,u)}{\partial x_n}|_{x^*,u^*} \end{bmatrix}$$

$$\nabla_u f(x^*, u^*) = \begin{bmatrix} \frac{\partial f_1(x,u)}{\partial u_1}|_{x^*,u^*} & \cdots & \frac{\partial f_1(x,u)}{\partial u_m}|_{x^*,u^*} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n(x,u)}{\partial u_1}|_{x^*,u^*} & \cdots & \frac{\partial f_n(x,u)}{\partial u_m}|_{x^*,u^*} \end{bmatrix}$$

Gain Scheduled LQR

Choose N way points in desired trajectory:

$$(x^*, u^*) \rightarrow \{(x_i^*, u_i^*)\}_{i=1}^N$$

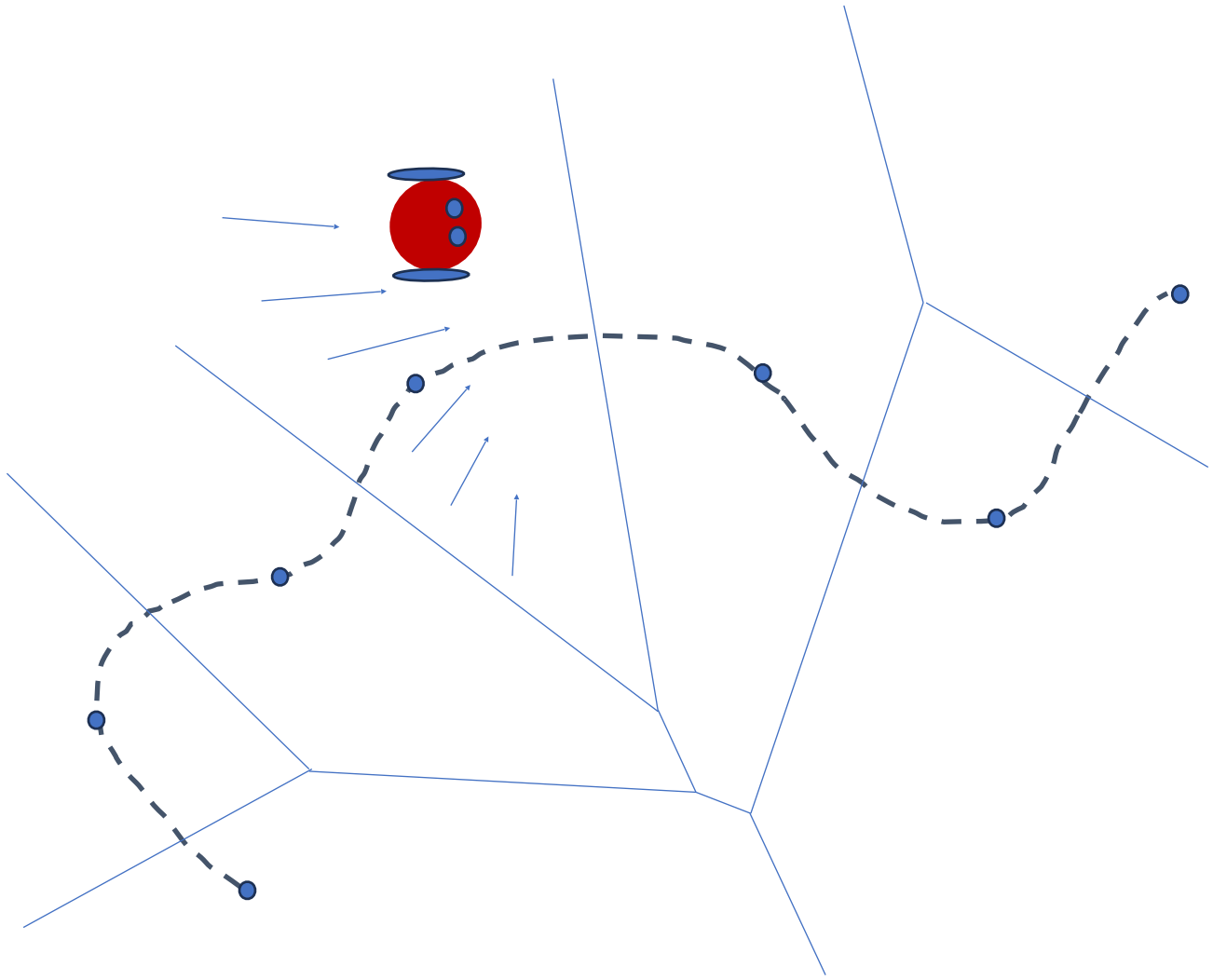
A collection of N LQR controllers:

$$\delta u = -K_i \delta x \quad \rightarrow \quad u = u^* - K_i(x - x^*)$$

$$K_i = R^{-1} B_i^T P_i$$

$$A_i^T P_i + P_i A_i - P_i B_i R^{-1} B_i^T P_i + Q = 0$$

Apply gain from closest point



Gain Scheduled LQR

Choose N way points in desired trajectory:

$$(x^*, u^*) \rightarrow \{(x_i^*, u_i^*)\}_{i=1}^N$$

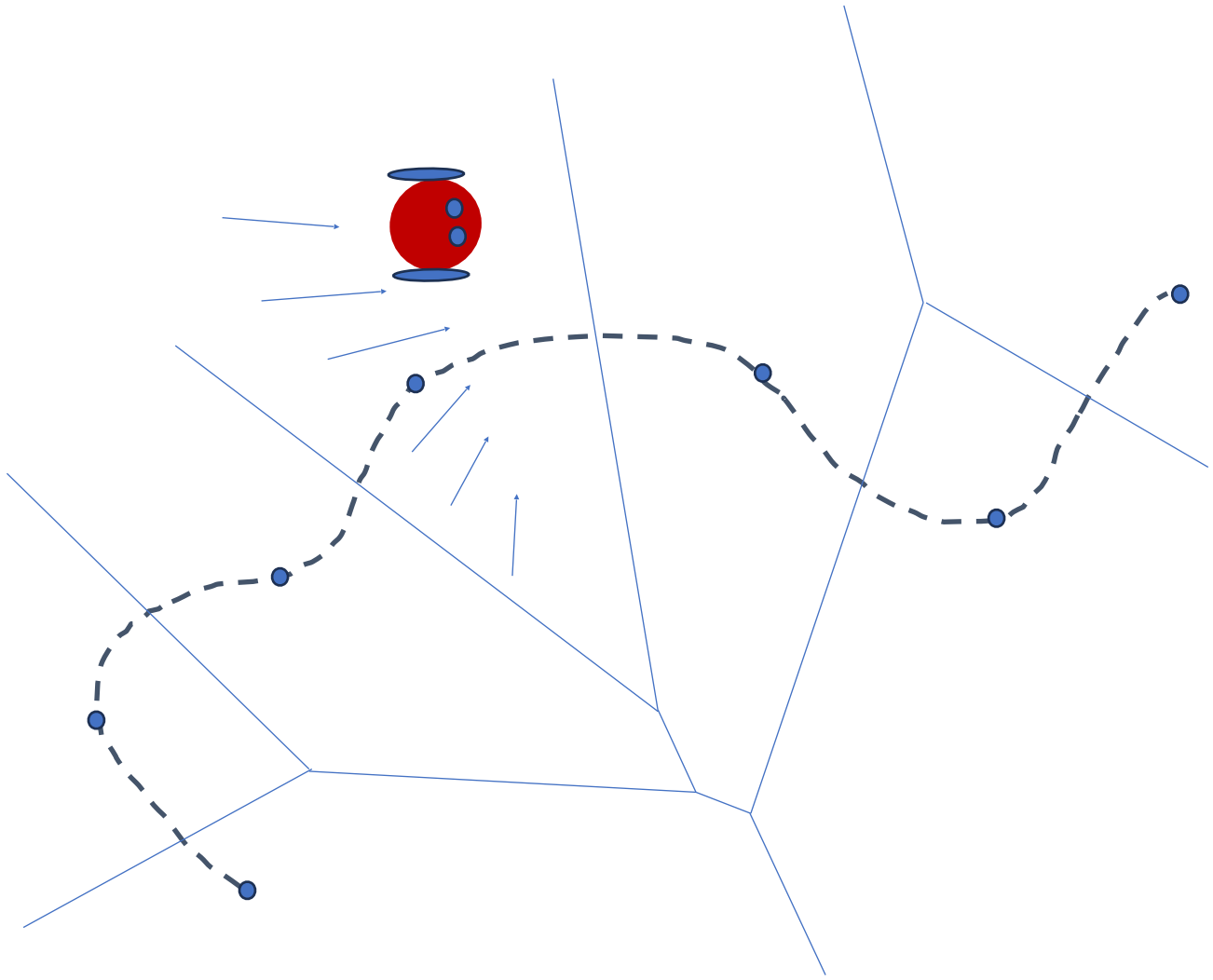
A collection of N LQR controllers:

$$\delta u = -K_i \delta x \quad \rightarrow \quad \boxed{u = u^* - K_i(x - x^*)}$$

$$K_i = R^{-1} B_i^T P_i$$

$$A_i^T P_i + P_i A_i - P_i B_i R^{-1} B_i^T P_i + Q = 0$$

Apply gain from closest point

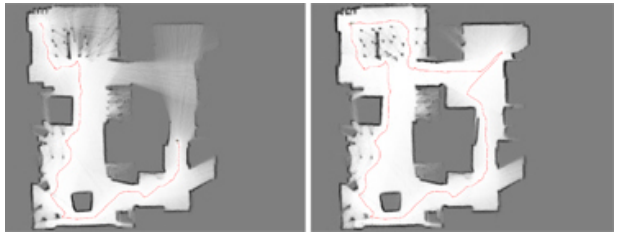


Perception Stack: Computer vision, filtering, SLAM

Use sensor data to update the models

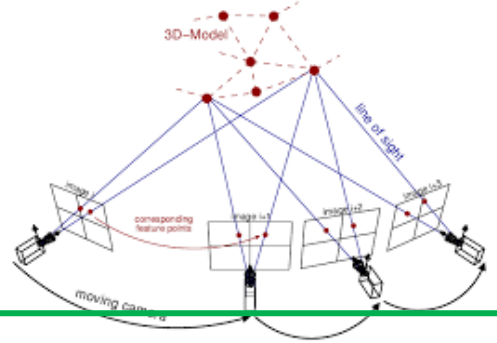
Localization, Mapping, Tracking:

- EKF/Monte Carlo localization
- Occupancy grid mapping
- Pose graph optimization
- Tracking (EKF and Particle Filter)



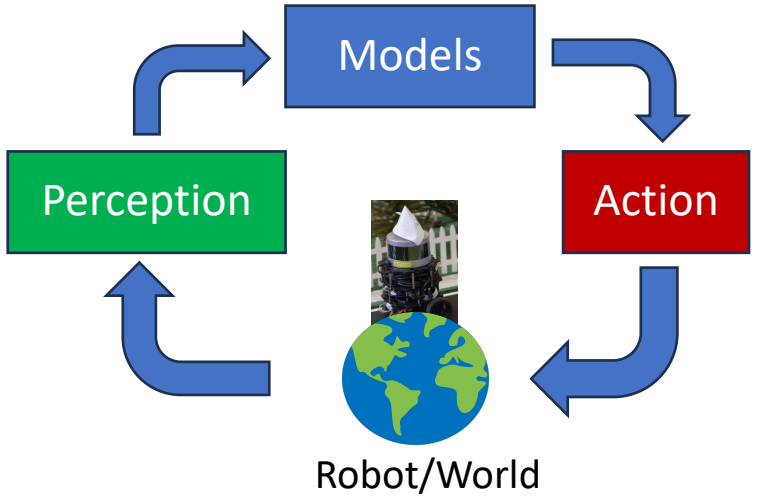
Information extraction

- Computer vision: features, correspondences, Structure from Motion (SfM), depth
- Lidar scan matching, ICP



Sensors:

- RGB Cameras, RGB-D/stereo cameras, Lidar
- IMU, GPS, wheel encoders



Next time: robotic sensors and introduction to computer vision